# “CAMPUS SMART-PARKING SYSTEM USING JAVA”

*Assignment report*

**CSA0922-PROGRAMMING IN JAVA**

*to the award of the degree of*

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE (AI)**


**Submitted by**

**OAVIKSHA P.L (192572001)**

**Under the Supervision of**

**Guide Name**

**Dr.VIJAYARAGAVAN**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences Chennai-602105**


**OCTOBER 2026**

# ABSTRACT

The Campus Smart-Parking System is designed to organize and simplify the parking activities of a university or college campus. It manages important operations such as vehicle entry and exit, parking-slot allocation, ticket generation, parking-fee calculation and electric-vehicle charging. By maintaining vehicle and parking information systematically, the proposed system aims to reduce manual work, improve the accuracy of parking records and make better use of the available parking space.

This report presents the system through a series of Java programming tasks. Each task focuses on a particular part of the design, from identifying the main objects and representing parking capacity to applying object-oriented programming principles. The examples demonstrate classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstraction, interfaces, encapsulation and data structures. The report also considers an extended design that supports electric bikes, delivery vans, one-day passes and five parking floors with unequal capacities. The code examples are educational demonstrations of individual features rather than one single, fully integrated application.

# 1. INTRODUCTION

A university parking facility is used daily by students, teaching and non-teaching staff, visitors and other authorized users. When parking is managed manually, it may be difficult to identify vacant spaces quickly, maintain accurate entry and exit records, avoid duplicate registrations and calculate the correct fee for each vehicle. These difficulties can lead to delays, confusion and inefficient use of the parking area.

The Campus Smart-Parking System addresses these needs by representing vehicles, parking floors, slots and tickets as separate software objects. When a vehicle arrives, the system can search for an available slot and associate that slot with the vehicle's registration number. During exit, it can validate the active parking record, calculate the applicable fee and make the slot available for another vehicle. Electric vehicles can also be charged an additional fee based on the amount of energy consumed.

Java's object-oriented features are suitable for this type of system because each class can have a clear responsibility. Common vehicle information can be placed in a parent class, while specific vehicle types can define their own rates or behaviours. This structure makes the design easier to understand, test and extend when the campus introduces new vehicle categories or pricing rules.

## 2. OBJECTIVES

The main objective of the project is to create a clear object-oriented model for common campus parking operations. The specific objectives are:

- Represent vehicles, floors, slots and tickets with Java classes.
- Allocate free slots and release them safely when vehicles exit.
- Calculate parking and electric charging fees.
- Protect object state using encapsulation and validation.
- Use inheritance, overriding, polymorphism, abstraction and interfaces.
- Extend the design for e-bikes, delivery vans, day passes and uneven floor capacities.

## 3. ASSUMPTIONS AND BUSINESS RULES

The following assumptions define the behaviour used in the report's examples. They provide a consistent basis for allocating parking spaces and calculating charges. A production system would need to validate these rules with the institution and implement additional checks for date, time and payment handling.

- Each active vehicle registration is unique.
- A slot holds only one vehicle at a time.
- Parking duration cannot be negative; hourly billing rounds up to a whole hour.
- Two-wheelers cost Rs 10/hour and cars cost Rs 30/hour.
- Electric-car charging costs Rs 12/kWh, added to parking fees.
- The Task 12 demonstration uses a flat Rs 25 e-bike parking fee.
- Delivery vans use twice the car rate during 9 AM–5 PM; the simple task example below illustrates the rate concept.
- The example one-day pass waives parking fees during validity; charging remains separate.

# 4. TASK-WISE CODE AND OUTPUT

This section explains the design concepts through separate Java examples. Each task includes a brief description of the problem, the relevant programming concept, the program code and its expected output. The examples should be considered individually because several tasks use a class named Main and are intended to be run as separate programs.

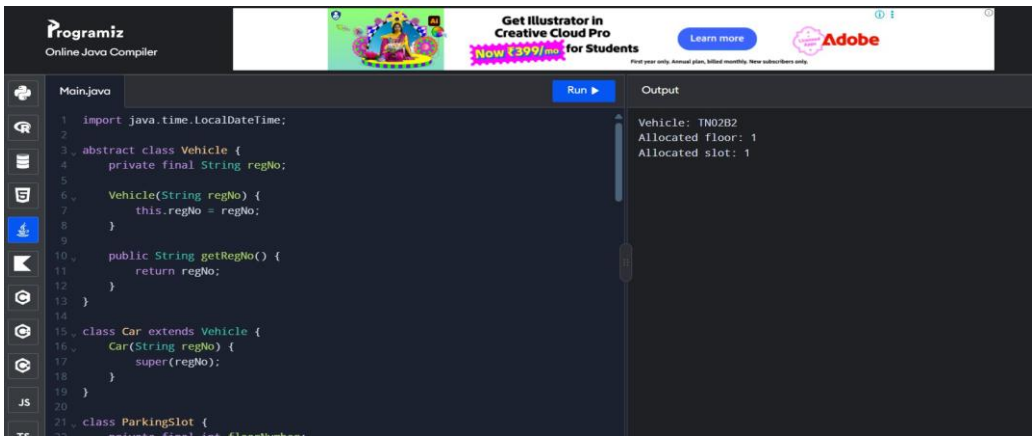
## Task 1 – Problem Analysis and Objects

The first step in developing the system is to identify the real-world entities and the information required to manage them. Candidate objects include Vehicle, ParkingLot, ParkingFloor, ParkingSlot and ParkingTicket. A vehicle stores details such as its registration number and category. A parking floor contains a collection of slots, while each slot represents one parking space. A parking ticket records the vehicle's active parking session and can be used when processing the exit.

The main inputs include the vehicle registration number, vehicle category, entry and exit times, and the amount of electricity consumed by a chargeable vehicle. Based on these inputs, the system can produce outputs such as the allocated slot, ticket information, parking duration and total fee. Identifying these objects before implementation helps separate the responsibilities of the system and makes later development more manageable.

The following example introduces a basic Vehicle class and a Car subclass. It demonstrates how a vehicle object can be created and how its registration number can be retrieved.

**Program code**



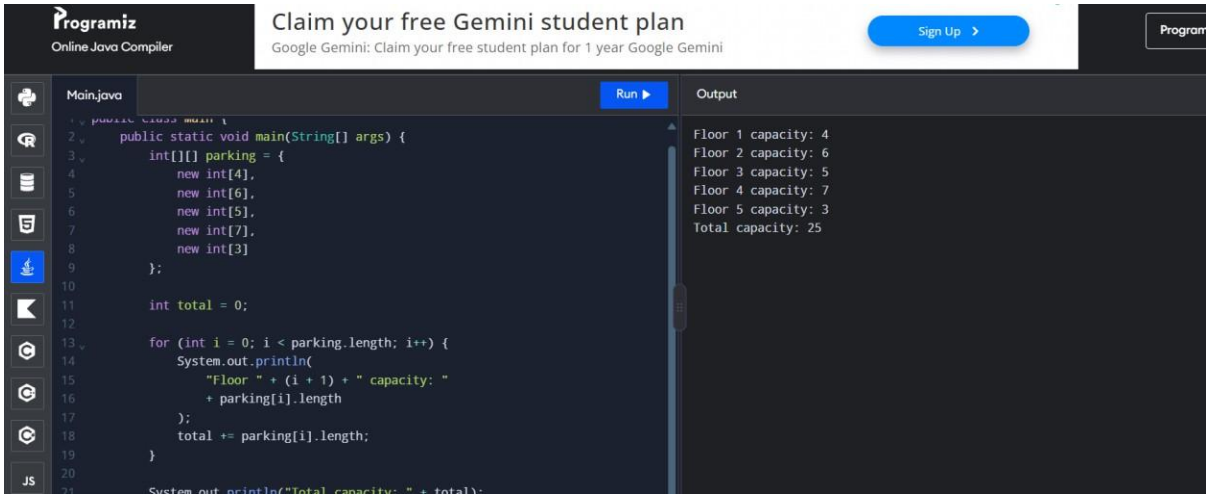
## Task 2 – Data Representation

A parking facility may have a different number of spaces on each floor. For example, some floors may be larger or may have fewer spaces because of ramps, pillars or other building features. A jagged array is useful for representing this arrangement because each row can have a different length. In this example, the five floors have capacities of 4, 6, 5, 7 and 3 slots.

The total capacity is calculated by adding the length of each row. The resulting total helps the parking operator understand how many vehicles the facility can accommodate when every slot is available. In a more complete implementation, the slot data would also track whether each space is occupied and which vehicle is using it.

A HashMap keyed by registration number can separately store active parking tickets or parking records. It supports expected constant-time lookup, insertion and removal under normal conditions, making it suitable for checking whether a vehicle is already parked. The array and map serve different purposes: the array represents physical parking capacity, while the map supports quick access to active vehicle records.

**Program code**




## Task 3 – Control Logic and Pseudocode

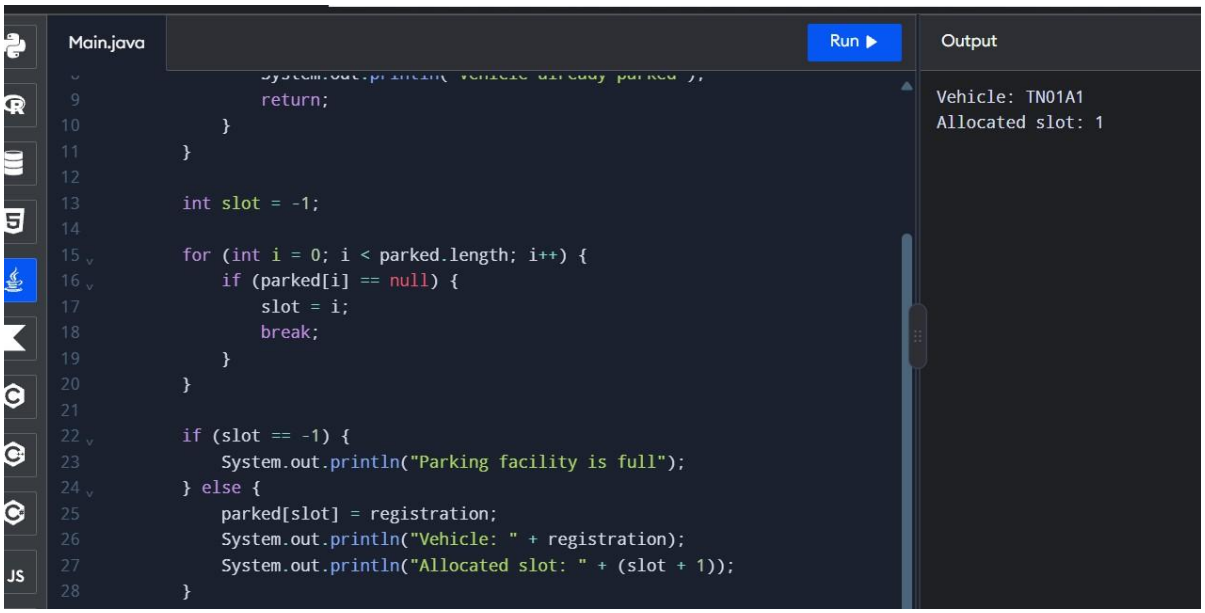
The control logic determines how the system responds to a vehicle entering or leaving the facility. During entry, the program scans the available slots in sequence and checks whether a slot is empty. Once a free slot is found, the vehicle's registration is stored in that position and the slot number is displayed. If every slot is occupied, the system should report that the facility is full rather than overwriting an existing vehicle record.

During exit, a complete system should locate the vehicle's active ticket, confirm that the ticket is valid, calculate the parking duration and fee, and then release the occupied slot. Invalid or

unknown registrations should be rejected without changing the state of another vehicle's record. Separating these steps helps prevent errors and ensures that slots are reused only after the vehicle has left.

The following example focuses on the allocation step using a small array of two slots. It illustrates the basic search-and-assign logic and the message displayed when a slot is successfully allocated.

**Program code**

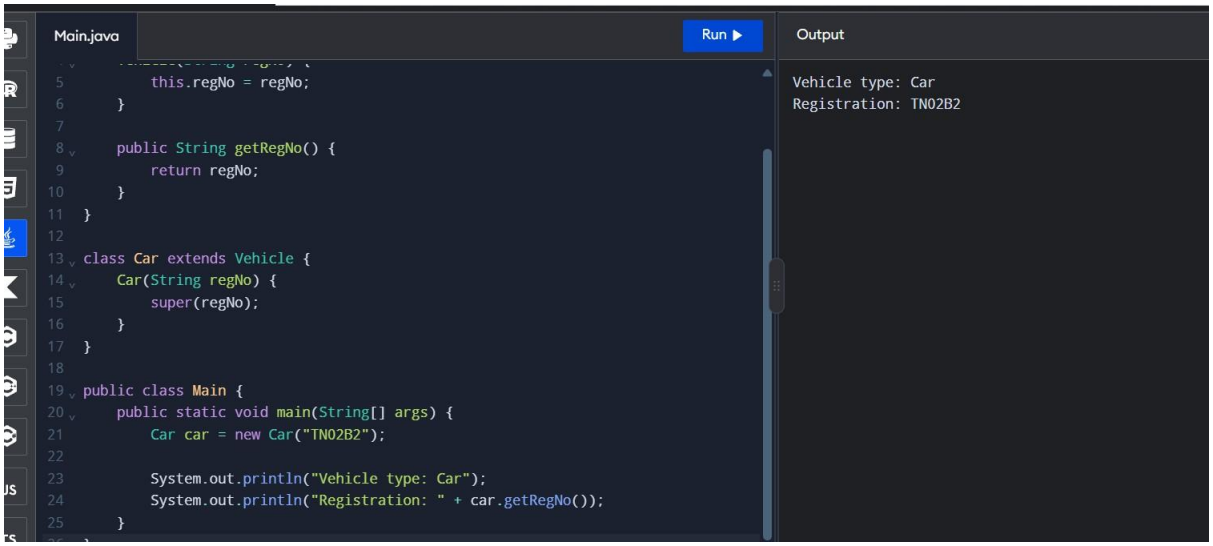



## Task 4 – Classes, Objects and Constructors

A class defines the data and behaviour that its objects will have, while an object is a particular instance of that class. In this example, Vehicle represents common vehicle information and Car represents a specific type of vehicle. The constructor is called when an object is created and is responsible for initializing its state.

The Car constructor calls super(regNo) to pass the registration number to the parent Vehicle constructor. This avoids repeating the same initialization logic in every child class. The registration field is declared private, so it cannot be accessed directly from unrelated code, and final, so its reference cannot be reassigned after initialization. The public getter provides controlled access to the value. These choices make the object's identity easier to protect and maintain.

**Program code**




## Task 5 – Static Members and Object Life Cycle

A static field belongs to the class itself rather than to an individual object. Therefore, all ParkingTicket objects share the same nextId counter. Whenever a ticket is created, the constructor assigns the current counter value to that ticket and then increments the counter. This allows each newly created ticket to receive a different sequential identifier.

Ticket identifiers are useful for distinguishing parking sessions and referring to a specific record during exit or support activities. In a complete system, the ticket would also contain information such as the vehicle registration number, entry time, assigned slot and payment status. The example focuses only on the counter and demonstrates that the two objects receive IDs 1 and 2.

**Program code**

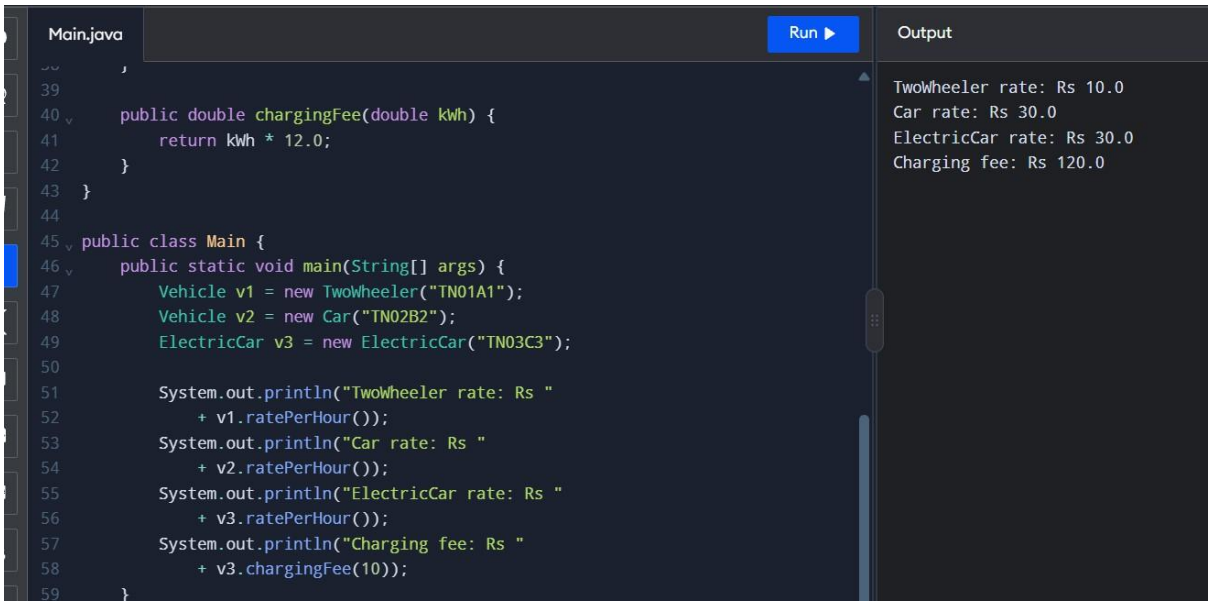


## Task 6 – Inheritance Hierarchy

Inheritance allows related vehicle classes to share a common structure. The abstract Vehicle class represents general vehicle behaviour, while TwoWheeler and Car provide their own hourly rates. This reduces duplication and gives the system a common type through which different vehicles can be handled.

An electric car can extend Car when it shares the properties and parking rate of an ordinary car, and it can implement a Chargeable interface to provide charging-related behaviour. This design keeps the general vehicle hierarchy separate from optional capabilities. The example below demonstrates two subclasses and their different hourly rates.

**Program code**




```
    public double chargingFee(double kWh) {
        return kWh * 12.0;
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new TwoWheeler("TN01A1");
        Vehicle v2 = new Car("TN02B2");
        ElectricCar v3 = new ElectricCar("TN03C3");

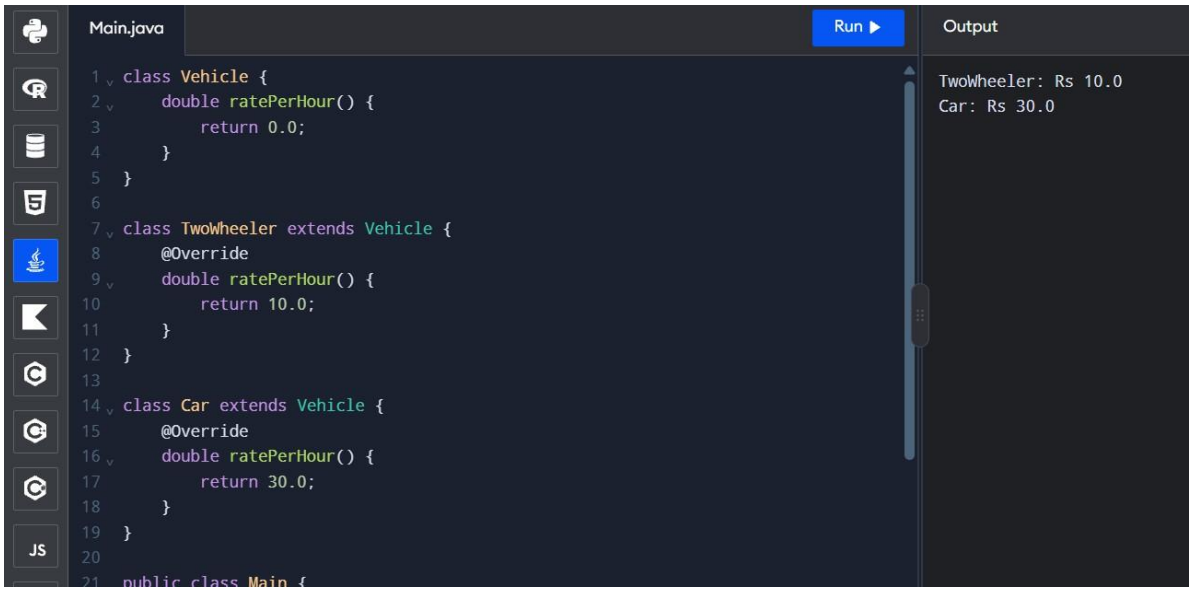
        System.out.println("TwoWheeler rate: Rs "
            + v1.ratePerHour());
        System.out.println("Car rate: Rs "
            + v2.ratePerHour());
        System.out.println("ElectricCar rate: Rs "
            + v3.ratePerHour());
        System.out.println("Charging fee: Rs "
            + v3.chargingFee(10));
    }
```

## Task 7 – Method Overriding

Method overriding occurs when a subclass provides its own implementation of a method already defined in its parent class. Here, Car overrides ratePerHour() and returns the car's hourly rate. The variable is declared using the parent type Vehicle, but it refers to a Car object.

When ratePerHour() is called, Java selects the implementation associated with the actual object at runtime. This behaviour is known as dynamic method dispatch and is one of the foundations of runtime polymorphism. It allows a parking system to calculate vehicle-specific charges without requiring every part of the application to contain separate type-checking logic.

**Program code**




## Task 8 – Polymorphism and Equality

Polymorphism allows a reference of a parent type to refer to objects belonging to different child classes. A Vehicle array can therefore store a mixture of cars, two-wheelers and other vehicle subclasses, provided they inherit from Vehicle. When an overridden method is called for each item, the correct implementation is selected for that object.

The example also demonstrates equality based on the registration number. Two vehicle objects with the same registration number are treated as equal by the supplied equals() method. The hashCode() method uses the same field, which is important when objects are used in hash-based collections. In a larger application, registration values should be validated and normalized consistently so that differences in letter case or extra spaces do not unintentionally create duplicate records.

**Program code**

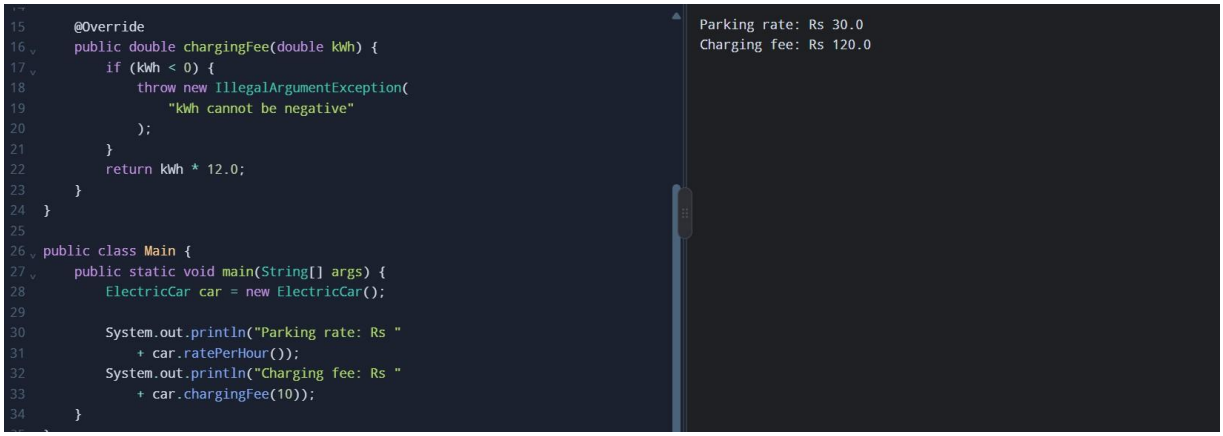


## Task 9 – Introduce Abstraction

Abstraction represents the essential features of an object while hiding details that do not need to be exposed to users of the class. The abstract Vehicle class defines a general requirement for a parking-rate calculation without prescribing one universal rate. Each concrete vehicle type must supply its own implementation.

The Chargeable interface describes an optional capability rather than a category of vehicle. An electric car can both inherit general vehicle behaviour and implement charging behaviour. This is useful because not every vehicle needs to support charging. In the example, the electric car charges Rs 12 per kilowatt-hour, so a consumption of 10 kWh produces a charging fee of Rs 120. In a complete application, this amount would be added to the parking fee.

**Program code**



```
    @Override
    public double chargingFee(double kWh) {
        if (kWh < 0) {
            throw new IllegalArgumentException(
                "kWh cannot be negative"
            );
        }
        return kWh * 12.0;
    }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar car = new ElectricCar();

        System.out.println("Parking rate: Rs "
            + car.ratePerHour());
        System.out.println("Charging fee: Rs "
            + car.chargingFee(10));
    }
```

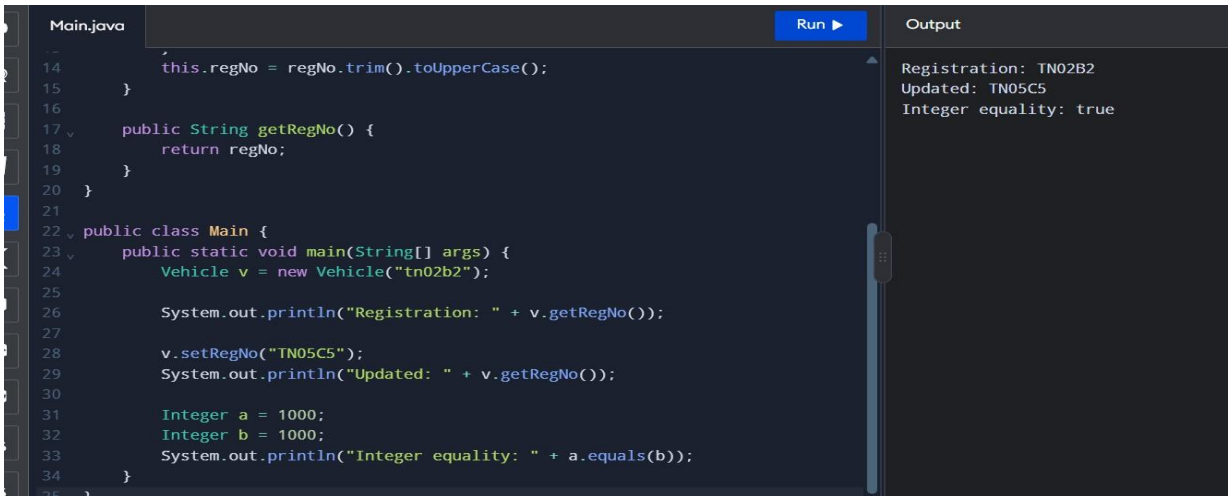
```
Parking rate: Rs 30.0
Charging fee: Rs 120.0
```

## Task 10 – Encapsulation, Packages and Access Modifiers

Encapsulation means keeping an object's internal data protected and allowing it to be accessed or changed through controlled methods. In this example, the registration number is private and is updated through setRegNo(). The setter checks that the supplied value is not null or blank, removes surrounding spaces and converts the registration to uppercase. This basic validation helps keep stored values consistent.

Access modifiers define where classes and members can be used. A private field is restricted to its own class, while a public method can be called by other parts of the application. As the project grows, related classes can be organized into packages such as parking.model, parking.pricing, parking.service and parking.app. Such organization makes responsibilities clearer and supports easier maintenance. When comparing wrapper objects, equals() should generally be used to compare values rather than ==, which checks whether two references point to the same object.

**Program code**



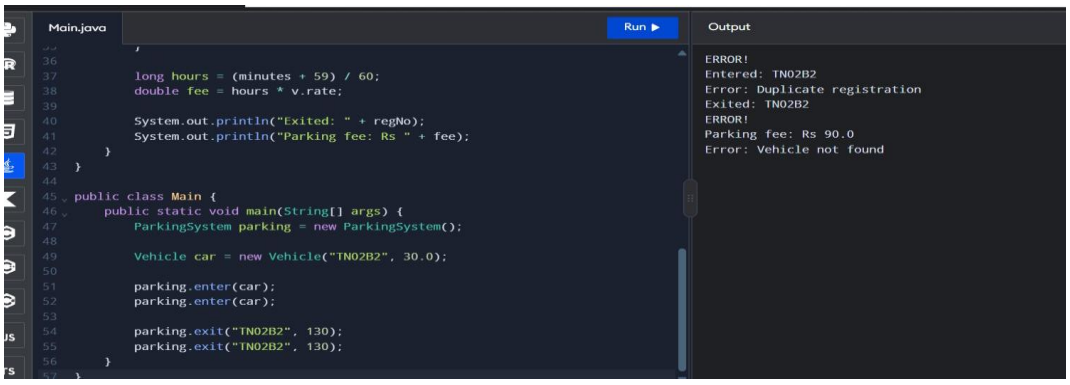

## Task 11 – Implementation, Testing and Defence

Testing checks whether the program behaves as expected for both normal and invalid operations. The supplied example tests a successful vehicle entry, a second entry using the same registration, a valid exit and an attempt to exit the same vehicle again. The duplicate registration is rejected, and the second exit is rejected because the active record has already been removed.

The fee calculation rounds a parking duration up to the next whole hour. For a stay of 130 minutes, the calculation produces three billable hours. At a car rate of Rs 30 per hour, the resulting fee is Rs 90. This rule should be tested at important boundaries, such as 0, 1, 60, 61, 120 and 121 minutes. Additional tests should cover a full facility, unknown registrations, invalid durations, charging fees and ticket/pass validity.

Testing should also confirm that an unsuccessful operation does not accidentally remove a valid record or occupy a slot incorrectly. These checks are important because parking systems maintain shared state and must keep the slot allocation and active-ticket records consistent.

**Program code**



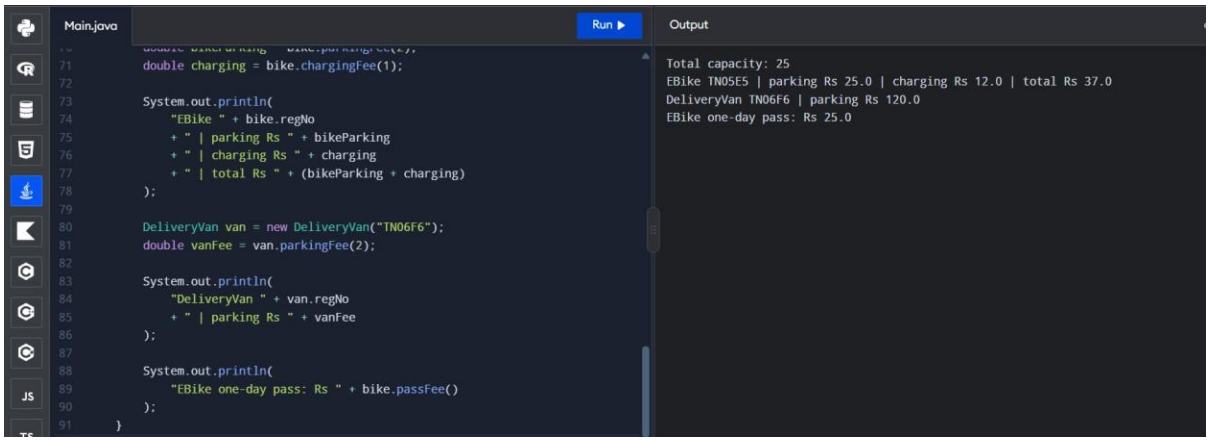
### Task 12 – Re-evaluate and Adapt the Design

The design can be extended to support new vehicle categories and campus-specific pricing rules without changing the overall structure. Unequal floor capacities can be represented with an array containing the capacity of each floor, while new vehicle types can be introduced as subclasses of the abstract Vehicle class. Each subclass can define the parking-fee behaviour that applies to it.

The e-bike example uses a flat parking fee of Rs 25, independent of the number of hours passed to the method. The delivery-van example uses Rs 60 per hour, which is twice the ordinary car rate of Rs 30 per hour. However, the code shown is a simplified demonstration: a complete implementation must check the actual parking times and apply the higher van rate only during the stated 9 AM to 5 PM period.

A one-day pass could be modelled as a separate pricing or pass component that checks whether the pass is valid for the vehicle and the parking period. According to the assumptions in this report, a valid pass waives parking charges during its validity, while electric charging remains a separate charge. These extensions illustrate how object-oriented design can accommodate changing requirements, although the full time-window and pass-validation logic is not implemented in this short code example.

**Program code**




# 5. COMPLEXITY ANALYSIS

Complexity analysis describes how the amount of work or memory used by an operation changes as the parking facility grows. Let **S** represent the total number of parking slots and **P** represent the number of vehicles currently parked. A sequential scan may inspect each slot before finding a free space, so its worst-case time complexity is O(S). If all slots are occupied, the scan may examine every slot before reporting that the facility is full.

A HashMap supports expected O(1) lookup, insertion and removal by registration number under normal hashing conditions. This makes it useful for maintaining active parking records. The slot data and active records together require O(S + P) space. These complexity values describe the data structures and operations shown in the report; actual performance can also depend on implementation details and the workload.

| Operation | Method / structure | Complexity |
|---|---|---|
| Find free slot | Sequential scan | O(S) worst case |
| Find active ticket | HashMap lookup | Expected O(1) |
| Insert active ticket | HashMap insertion | Expected O(1) |
| Remove active ticket | HashMap removal | Expected O(1) |
| Storage | Slots and active tickets | O(S + P) space |

S = total number of parking slots; P = number of active parked vehicles.

## 6. TEST CASE SUMMARY

The following test cases summarize important expected behaviours of the proposed system. They cover normal fee calculation, duplicate prevention, validation during exit, unequal floor capacities, electric charging, delivery-van pricing and one-day pass handling. The expected results are based on the assumptions stated earlier in the report.

These cases can be used as a starting point for manual testing or for creating automated Java unit tests. The implementation should be tested with both valid and invalid inputs, and the actual output should be compared with the expected result. Some cases describe requirements for the extended design rather than functionality fully implemented in the short code snippets.

| Test | Scenario | Expected result | Purpose |
|---|---|---|---|
| TC01 | Two-wheeler for 130 minutes | Rs 30 | Three billable hours |
| TC02 | Car for 130 minutes | Rs 90 | Three billable hours |
| TC03 | Electric car, 130 minutes and 10 kWh | Rs 210 total | Parking plus charging |
| TC04 | Duplicate active registration | Rejected | Prevent duplicate parking |
| TC05 | Unknown registration at exit | Rejected | Validate active ticket |
| TC06 | Five floor capacities 4,6,5,7,3 | 25 slots | Unequal floor support |
| TC07 | E-bike with 1 kWh charging | Rs 37 in example | Flat fee plus charging |
| TC08 | Van for two peak hours | Rs 120 | Double-car rate example |
| TC09 | Valid one-day pass | Parking fee waived | Pass validation |

# 7. CONCLUSION

The Campus Smart-Parking System provides an example of how Java and object-oriented programming can be applied to a practical campus management problem. The report identifies the main entities involved in parking, represents parking floors with unequal capacities, and demonstrates basic slot allocation and active-vehicle tracking. It also explains how parking fees and electric-vehicle charging fees can be calculated according to defined business rules.

The examples demonstrate several important Java concepts. Classes and constructors are used to create and initialize objects; static members generate sequential ticket identifiers; inheritance allows vehicle types to share common behaviour; and method overriding and polymorphism support vehicle-specific rates. Abstraction defines common requirements, interfaces represent optional charging capabilities, and encapsulation with validation helps protect object data. Arrays and hash-based collections support the representation of parking spaces and active records.

The extended design considers e-bikes, delivery vans, one-day passes and floors with different capacities. These additions show that a well-structured object-oriented design can be adapted as requirements change. The code snippets are demonstrations of individual concepts and do not implement every feature as one integrated application. Further development could combine the classes into a complete system, add robust date-and-time handling, validate all inputs, persist records in a database, manage payments, and test peak-hour rates and pass validity. Overall, the project provides a useful foundation for understanding how software design principles can improve the organization, accuracy and maintainability of campus parking operations.